

PROJECT TỔNG KẾT OOP TRONG C#

- MỤC TIÊU DỰ ÁN

Xây dựng Hệ thống Quản lý Thiết bị Công nghiệp (Industrial Device Management System) bằng Console Application, áp dụng đầy đủ 4 tính chất của OOP:

Tính chất	Áp dụng trong project
Encapsulation (Đóng gói)	Private fields, Public properties, Validation trong setter
Inheritance (Kế thừa)	Class cha <code>Device</code> , các class con <code>Sensor</code> , <code>Motor</code> , <code>PLC</code>
Polymorphism (Đa hình)	Override method <code>GetStatus()</code> , <code>Start()</code> , <code>Stop()</code>
Abstraction (Trừu tượng)	Abstract class <code>Device</code> , Interface <code>IControllable</code>

- ĐỀ BÀI CHI TIẾT

PHẦN 1: TẠO CẤU TRÚC CLASS

1.1. Tạo Interface `IControllable`

Interface này định nghĩa các hành động điều khiển mà thiết bị có thể thực hiện.

Yêu cầu:

- Method `Start()` - Khởi động thiết bị, trả về `bool`
- Method `Stop()` - Dừng thiết bị, trả về `bool`
- Method `Reset()` - Reset thiết bị, trả về `void`

1.2. Tạo Abstract Class `Device` (Class cha)

Đây là class cơ sở cho tất cả thiết bị, implement interface `IControllable`.

Yêu cầu về Fields (private):

- `_deviceId` : string - Mã thiết bị
- `_deviceName` : string - Tên thiết bị

- `_location` : string - Vị trí lắp đặt
- `_isRunning` : bool - Trạng thái đang chạy hay không
- `_installDate` : DateTime - Ngày lắp đặt

Yêu cầu về Properties (public):

- Tất cả fields trên đều có Property tương ứng
- Property `DeviceId` : chỉ cho phép **đọc** (get), không cho sửa từ bên ngoài
- Property `DeviceName` : có **validation** - không được để trống
- Property `IsRunning` : chỉ cho phép **đọc** từ bên ngoài

Yêu cầu về Constructor:

- Nhận 3 tham số: `deviceId` , `deviceName` , `location`
- Tự động gán `_installDate = DateTime.Now`
- Tự động gán `_isRunning = false`

Yêu cầu về Methods:

- `GetStatus()` : **abstract** method, trả về `string` - Mỗi loại thiết bị sẽ override khác nhau
 - `GetInfo()` : **virtual** method, trả về `string` - Thông tin cơ bản của thiết bị
 - `Start()` : Implement từ interface, set `_isRunning = true` , return `true`
 - `Stop()` : Implement từ interface, set `_isRunning = false` , return `true`
 - `Reset()` : Implement từ interface, gọi `Stop()` rồi in ra "Device reset"
-

1.3. Tạo Class `Sensor` (Kế thừa từ `Device`)

Đại diện cho các loại cảm biến (nhiệt độ, áp suất, lưu lượng...).

Yêu cầu về Fields riêng:

- `_sensorType` : string - Loại cảm biến (Temperature, Pressure, Flow, Level...)
- `_currentValue` : double - Giá trị đang đo được
- `_unit` : string - Đơn vị (°C, bar, m³/h, mm...)
- `_minValue` : double - Giá trị tối thiểu
- `_maxValue` : double - Giá trị tối đa

Yêu cầu về Properties:

- Tất cả fields có Property tương ứng

- Property `CurrentValue` : có **validation** - giá trị phải nằm trong khoảng `[MinValue, MaxValue]`

Yêu cầu về Constructor:

- Gọi constructor của class cha (base)
- Nhận thêm các tham số: `sensorType`, `unit`, `minValue`, `maxValue`
- Gán `_currentValue = 0`

Yêu cầu về Methods:

- **Override** `GetStatus()` : Trả về chuỗi dạng `"[SensorType] Value: currentValue unit (Running/Stopped)"`
 - **Override** `GetInfo()` : Gọi `base.GetInfo()` + thêm thông tin riêng của Sensor
 - `ReadValue()` : Giả lập đọc giá trị ngẫu nhiên trong khoảng `[MinValue, MaxValue]`
 - `IsAlarm()` : Trả về `true` nếu giá trị vượt 80% của `MaxValue`
-

1.4. Tạo Class `Motor` (Kế thừa từ `Device`)

Đại diện cho động cơ, bơm, quạt...

Yêu cầu về Fields riêng:

- `_ratedPower` : double - Công suất định mức (kW)
- `_ratedSpeed` : int - Tốc độ định mức (RPM)
- `_currentSpeed` : int - Tốc độ hiện tại (RPM)

Yêu cầu về Properties:

- Tất cả fields có Property tương ứng
- Property `CurrentSpeed` : có **validation** - không được âm và không vượt quá 120% `RatedSpeed`

Yêu cầu về Constructor:

- Gọi constructor của class cha (base)
- Nhận thêm các tham số: `ratedPower`, `ratedSpeed`
- Gán `_currentSpeed = 0`

Yêu cầu về Methods:

- **Override** `GetStatus()` : Trả về `"Motor [name]: currentSpeed/ratedSpeed RPM (Running/Stopped)"`

- **Override** `GetInfo()` : Gọi `base.GetInfo()` + thêm thông tin công suất, tốc độ
 - **Override** `Start()` : Gọi `base.Start()` + set `CurrentSpeed = RatedSpeed`
 - **Override** `Stop()` : Gọi `base.Stop()` + set `CurrentSpeed = 0`
 - `SetSpeed(int speed)` : Đặt tốc độ cho motor, có validation
-

1.5. Tạo Class `PLC` (Kế thừa từ `Device`)

Đại diện cho bộ điều khiển PLC.

Yêu cầu về Fields riêng:

- `_brand` : string - Hãng sản xuất (Siemens, Mitsubishi, Omron...)
- `_model` : string - Model (S7-1200, FX5U, CP1H...)
- `_ipAddress` : string - Địa chỉ IP
- `_isConnected` : bool - Trạng thái kết nối

Yêu cầu về Properties:

- Tất cả fields có Property tương ứng
- Property `IpAddress` : có **validation** - kiểm tra định dạng IP đơn giản (chứa 3 dấu chấm)

Yêu cầu về Constructor:

- Gọi constructor của class cha (base)
- Nhận thêm các tham số: `brand` , `model` , `ipAddress`
- Gán `_isConnected = false`

Yêu cầu về Methods:

- **Override** `GetStatus()` : Trả về `"PLC [brand model]: Connected/Disconnected - Running/Stopped"`
 - **Override** `GetInfo()` : Gọi `base.GetInfo()` + thêm thông tin IP, brand, model
 - `Connect()` : Set `_isConnected = true` , in thông báo
 - `Disconnect()` : Set `_isConnected = false` , in thông báo
 - **Override** `start()` : Chỉ start được khi đã Connect, nếu chưa connect thì return `false`
-

1.6. Tạo Class `DeviceManager`

Class quản lý danh sách tất cả thiết bị.

Yêu cầu về Fields:

- `_devices` : `List<Device>` - Danh sách thiết bị

Yêu cầu về Methods:

- `AddDevice(Device device)` : Thêm thiết bị vào danh sách
- `RemoveDevice(string deviceId)` : Xóa thiết bị theo ID
- `FindDevice(string deviceId)` : Tìm thiết bị theo ID, trả về `Device` hoặc `null`
- `GetAllDevices()` : Trả về danh sách tất cả thiết bị
- `GetDevicesByType<T>()` : Trả về danh sách thiết bị theo loại (dùng Generic)
- `StartAllDevices()` : Khởi động tất cả thiết bị
- `StopAllDevices()` : Dừng tất cả thiết bị
- `PrintAllStatus()` : In trạng thái tất cả thiết bị (dùng **Polymorphism** - gọi `GetStatus()`)

PHẦN 2: VIẾT CHƯƠNG TRÌNH CHÍNH (Program.cs)

Tạo menu console với các chức năng:

```
=====
INDUSTRIAL DEVICE MANAGEMENT SYSTEM
=====
1. Hiển thị tất cả thiết bị
2. Hiển thị trạng thái thiết bị
3. Thêm thiết bị mới
4. Tìm kiếm thiết bị theo ID
5. Khởi động thiết bị
6. Dừng thiết bị
7. Khởi động tất cả
8. Dừng tất cả
9. Đọc giá trị Sensor
10. Đặt tốc độ Motor
11. Kết nối/Ngắt PLC
0. Thoát
=====
Chọn chức năng: _
```

PHẦN 3: KẾT QUẢ MONG ĐỢI

3.1. Khi chọn "1. Hiển thị tất cả thiết bị":

=== DANH SÁCH THIẾT BỊ ===

[1] SENSOR - SEN001

Tên: Temperature Sensor 1
Vị trí: Production Line 1
Loại: Temperature | Đơn vị: °C
Khoảng đo: 0 - 100
Ngày lắp: 15/01/2024

[2] MOTOR - MOT001

Tên: Conveyor Motor 1
Vị trí: Production Line 1
Công suất: 5.5 kW | Tốc độ định mức: 1450 RPM
Ngày lắp: 15/01/2024

[3] PLC - PLC001

Tên: Main PLC
Vị trí: Control Room
Hãng: Siemens S7-1200
IP: 192.168.1.10
Ngày lắp: 15/01/2024

Tổng: 3 thiết bị

3.2. Khi chọn "2. Hiện thị trạng thái":

=== TRẠNG THÁI THIẾT BỊ ===

SEN001: [Temperature] Value: 25.5 °C (Stopped)
MOT001: Motor Conveyor Motor 1: 0/1450 RPM (Stopped)
PLC001: PLC Siemens S7-1200: Disconnected - Stopped

3.3. Khi khởi động Motor:

Nhập ID thiết bị: MOT001
>> Motor Conveyor Motor 1 đã khởi động!
>> Tốc độ hiện tại: 1450 RPM

3.4. Khi đọc giá trị Sensor:

```
Nhập ID Sensor: SEN001
>> Đang đọc giá trị...
>> Giá trị đọc được: 45.7 °C
>> Trạng thái: Bình thường
```

- CẤU TRÚC THƯ MỤC PROJECT

```
IndustrialDeviceManagement/
|
|— Interfaces/
|   |— IControllable.cs
|
|— Models/
|   |— Device.cs           (Abstract class)
|   |— Sensor.cs          (Kế thừa Device)
|   |— Motor.cs           (Kế thừa Device)
|   |— PLC.cs             (Kế thừa Device)
|
|— Services/
|   |— DeviceManager.cs
|
|— Program.cs
```

- CHECKLIST ĐÁNH GIÁ

Học viên tự kiểm tra đã hoàn thành các yêu cầu sau:

Encapsulation (Đóng gói)

- ☐ Tất cả fields đều là `private`
- ☐ Sử dụng Properties để truy cập fields
- ☐ Có validation trong setter của ít nhất 3 properties
- ☐ Có property chỉ cho đọc (get only)

Inheritance (Kế thừa)

- ☐ Class `Sensor`, `Motor`, `PLC` kế thừa từ `Device`
- ☐ Sử dụng từ khóa `base` để gọi constructor cha

- ☐ Sử dụng `base.MethodName()` để gọi method của class cha

Polymorphism (Đa hình)

- ☐ Method `GetStatus()` được override ở tất cả class con
- ☐ Method `GetInfo()` được override và gọi `base.GetInfo()`
- ☐ `DeviceManager.PrintAllStatus()` gọi `GetStatus()` và mỗi loại thiết bị hiển thị khác nhau

Abstraction (Trừu tượng)

- ☐ Class `Device` là abstract class
- ☐ Method `GetStatus()` là abstract method
- ☐ Interface `IControllable` được implement bởi class `Device`

Khác

- ☐ Code có comment giải thích đầy đủ
- ☐ Chương trình chạy không lỗi
- ☐ Menu console hoạt động đúng

- GỢI Ý TRIỂN KHAI

Thứ tự code:

1. Tạo `IControllable.cs` trước
2. Tạo `Device.cs` (abstract class)
3. Tạo `Sensor.cs`, `Motor.cs`, `PLC.cs`
4. Tạo `DeviceManager.cs`
5. Viết `Program.cs`

Lưu ý:

- Mỗi file một class/interface
 - Comment bằng tiếng Việt cho dễ hiểu
 - Test từng class trước khi ghép lại
-

- KIẾN THỨC CẦN NHỚ

1. Cú pháp Abstract Class & Method

```
public abstract class Device
{
    public abstract string GetStatus(); // Không có body
}
```

2. Cú pháp Override

```
public override string GetStatus()
{
    return "..."; // Phải có body
}
```

3. Cú pháp Interface

```
public interface IControllable
{
    bool Start();
    bool Stop();
}
```

4. Cú pháp Property với Validation

```
private string _name;
public string Name
{
    get { return _name; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Name không được trống!");
        _name = value;
    }
}
```

5. Gọi Constructor cha

```
public Sensor(string id, string name, string location, string sensorType)
    : base(id, name, location) // Gọi constructor của Device
{
    _sensorType = sensorType;
}
```